An Inductive Logic Programming Approach for ARC-AGI-3

by

Abu Safwan Md Farhan
22201340
Zihadul Islam Tasin
23301690
K. M. Mehrab Hossain
22301468
Iftekhar Mohammed Aritro
22299470

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
September 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing our degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Abu Safwan Md Farhan	Zihadul Islam Tasin
22201340	23301690
K. M. Mehrab Hossain	Iftekhar Mohammed Aritro
22301468	22299470

Approval

The thesis titled "An Inductive Logic Programming Approach for ARC-AGI-3" submitted by

1. Abu Safwan Md Farhan (22201340)
2. Zihadul Islam Tasin (23301690)
3. K. M. Mehrab Hossain (22301468)
4. Iftekhar Mohammed Aritro (22299470)

of Summer, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in 2026.

Examining Committee:

Supervisor:
(Member)



Dr. Chowdhury Mofizur Rahman

Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Utsha Kumar Roy

Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor & Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

ARC-AGI-3, an interactive benchmark consisting of 135 games that have no explicitly stated win-condition, has become one of the most important benchmarks for measuring frontier AI intelligence in recent times. Even though humans excel at these games, state-of-the-art AI performance remains subpar. Because of the extreme sparseness of reward in these games, Reinforcement Learning approaches, the de facto paradigm for goal-driven interactive tasks, may perform poorly. Under such reward-sparse conditions, we explore Inductive Logic Programming (ILP), a type of Machine Learning, as the alternative. Although ILP has been tested on previous iterations of ARC-AGI, its application to ARC-AGI-3 remains limited. We first show why some Reinforcement Learning systems may fail on ARC-AGI-3 and then show how ILP performs on this benchmark.

Keywords: ARC-AGI-3, Inductive Logic Programming, Reinforcement Learning

Dedication

To our parents, and to all the people of the world who have fought against injustice.

Acknowledgement

We would like to thank our co-supervisor Utsha Kumar Roy for the incredible kindness he showed us throughout our thesis journey. His correspondence helped us organize our thoughts, and finally led us to our final thesis.

We would also like to thank Md Humaion Kabir Mehedi for his kindness, for being our mentor for this journey, and for providing us with all sorts of logistical help.

Most importantly, we would like to thank our supervisor Dr. Chowdhury Mofizur Rahman for treating us with extreme patience and kindness and for giving us immense intellectual freedom. Our regular meetings kept us on our toes and helped us finish our work on time.

Finally, we would like to thank all of the members of faculty for providing us with knowledge.

Table of Contents

Declaration i

Approval ii

Abstract iv

Dedication v

Acknowledgment vi

Table of Contents vii

List of Figures ix

List of Tables x

Nomenclature xii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Motivation . . . 2
- 1.3 Problem Statement . . . 2
  - 1.3.1 Goal Discovery . . . 3
  - 1.3.2 World Model Induction . . . 3
  - 1.3.3 Efficiency . . . 3
- 1.4 Objective . . . 3
- 1.5 Methodology in Brief . . . 4
- 1.6 Scope and Limitations . . . 4
  - 1.6.1 Scope . . . 4
  - 1.6.2 Limitations . . . 5
- 1.7 Layout of the Dissertation . . . 5

2 Literature Review 6
- 2.1 Preliminaries . . . 6
  - 2.1.1 ARC-AGI-3 . . . 6
  - 2.1.2 Reinforcement Learning . . . 8
  - 2.1.3 Inductive Logic Programming . . . 8
- 2.2 Review of Existing Research . . . 10
  - 2.2.1 ARC-AGI-3 Preview Agent Competition . . . 10

2.2.2 Symbolic Reinforcement Learning using Inductive Logic Programming . . . 12
2.2.3 The ILASP System for Inductive Learning of Answer Set Programs . . . 12
2.2.4 Duck Harness . . . 13
2.2.5 Deep Q-Networks . . . 14
2.2.6 Summary of Key Findings . . . 15
2.2.7 Failure Analysis of Existing Approaches . . . 16

3 Requirements, Impacts and Constraints 17
3.1 Final Specifications and Requirements . . . 17
3.1.1 Technical and Methodological Requirements . . . 17
3.1.2 Software Requirements . . . 17
3.2 Societal Impact . . . 17
3.3 Environmental Impact . . . 18
3.4 Ethical Issues . . . 18
3.5 Standards . . . 19
3.6 Project Management Plan . . . 19
3.6.1 Resource Management . . . 19
3.7 Risk Management . . . 19
3.8 Economic Analysis . . . 20

4 Proposed Methodology 21
4.1 Methodology Overview . . . 21
4.2 RL vs. Random Agent Methodology . . . 21
4.3 PermanentLearner Methodology . . . 22
4.4 PermanentLearner Experimental Setup on 3 Toy Examples . . . 23
4.5 InteractivePopper . . . 24
4.5.1 Predefined Assumptions and Background Knowledge . . . 24
4.5.2 InteractivePopper Workflow . . . 24

5 Result Analysis 26
5.1 Performance Evaluation . . . 26
5.2 RL vs. Random Agent Results . . . 26
5.3 PermanentLearner Initial Results . . . 26
5.4 Analysis of Design Solutions . . . 29
5.5 Final Design Adjustments . . . 30
5.6 Statistical Analysis . . . 31
5.7 Comparisons and Relationships . . . 32
5.8 InteractivePopper Results . . . 33
5.9 Discussions . . . 34

6 Conclusion 36
6.1 Summary of Findings . . . 36
6.2 Contributions to the Field . . . 37
6.3 Recommendations for Future Work . . . 37

Bibliography 41

List of Figures

4.1 InteractivePopper Workflow 24

5.1 Sample efficiency on ARC-AGI-3 (`ls20` Level 1): Cumulative level completions over 60,000 environment interactions comparing the Random Agent Baseline against the DQN Agent across 5 independent random seeds. The solid lines represent empirical means, and the shaded regions indicate ±1 standard deviation. 27

List of Tables

1.1 Problem decomposition: what each challenge requires and what this paper tests. 3

2.1 Comparison of ARC-AGI-3 Preview Agent Competition approaches. . 15

3.1 Risk Register 19

4.1 Hyperparameter Specifications and Network Architecture for DQN on ARC-AGI-3 21

5.1 Comparative Performance Across 5 Independent Seeds on ARC-AGI `ls20` (Level 1, 60,000 Actions) 27
5.2 Verified ARC-AGI Online Recordings on `ls20` (Human Baseline $H =$ 22 Actions) 28
5.3 PermanentLearner vs. Q-learning baseline on `aaa_small` (30 seeds) . 28
5.4 Hidden-goal induction and planning results, 10 seeds per game 29
5.5 InteractivePopper performance in public ARC-AGI-3 games 33
5.6 InteractivePopper prediction accuracy 34

6.1 Summary of Contributions 37

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

A Set of all admissible Actions

$P(s' \mid s, a)$ Transition Probability

$R(s, a)$ Reward Function

S Set of all possible States

γ Discount Factor

AGI Artificial General Intelligence

AI Artificial Intelligence

ALE Arcade Learning Environment

API Application Programming Interface

ARC-AGI-3 Abstraction and Reasoning Corpus-Artificial General Intelligence 3

ASP Answer Set Programming

CDPI Context-Dependent Partial Interpretation

CNN Convolutional Neural Network

CPU Central Processing Unit

DQN Deep Q-Network

GPU Graphics Processing Unit

ILP Inductive Logic Programming

IMO International Mathematical Olympiad

LFF Learning from Failures

LLM Large Language Model

MDL Minimum Description Length

MDP Markov Decision Process

REPL Read-Eval-Print Loop

RHAE Relative Human Action Efficiency

RL Reinforcement Learning

VGDL Video Game Description Language

Chapter 1

Introduction

1.1 Background

We are much closer to AGI[1] now than we were at the inception of the AI field in 1956 [1]. In 2016, Lee Sedol, the world's second-best Go player at the time, was defeated by AlphaGo [6] (whose successor MuZero [11] is even stronger). AlphaFold led to Hassabis and Jumper winning the 2024 Nobel Prize in Chemistry for protein structure prediction [15], [18], and Gemini [23] and others [22] have achieved gold-medal standard at the IMO.

Although the breakthroughs listed above are groundbreaking, AGI is yet to be realized. The current AI frontier is *jagged* [17]. AI demonstrates exceptional behavior in things very difficult for most humans (e.g., 9-dan-level Go, protein structure prediction, IMO-level math), but is unable to do easier things (e.g., performing a long sequence of daily household activities like cleaning). This is not a new phenomenon: to quote Pinker from 1994, "But household robots are still confined to science fiction. The main lesson of thirty-five years of AI research is that the hard problems are easy and the easy problems are hard." [4, p. 192] Our job as AI researchers is to make the AI frontier smooth and even by eliminating the problems holding us down one by one.

To measure this difference, François Chollet and Mike Knoop came up with an important benchmark: the ARC-AGI-3. It was published by the ARC Prize Foundation in March 2026 [27].

ARC-AGI-3 is the third version of the ARC-AGI series and the hardest version. The first two types, ARC-AGI-1 [16] and ARC-AGI-2 [20], were static puzzle-based benchmarks. There, the agent was asked to understand a pattern by being shown a fixed grid. But ARC-AGI-3 is totally different. This is the first fully interactive reasoning benchmark. Here, the agent is left in a novel game environment, without any instructions, without any rules, and without any stated goal. The agent has to understand how the environment is working by exploring, has to understand the goal, and take action according to that.

Humans can score 100% on this benchmark. But as of June 2026, the most powerful frontier AI systems—including Claude Opus 4.6—scored below 1.5% on the leaderboard snapshot recorded at that date [28]. This gap still exposes a fundamental limitation of current AI.

[1] We use the term here informally. For a more formal treatment, see [24]

1.2 Motivation

Humans have the ability to adapt to any new situations. A person enters into a new environment and observes, explores, and soon understands what is going on. They develop their understanding of rules and begin to develop an objective. This capability is referred to as quick adaptation.

ARC-AGI-3 is a test of exactly this ability. It embeds an AI agent within a game setting that is different from the one the agent has previously been trained on. No rules or instructions, no goal. The agent has to explore the environment by itself and learn what the goal is and how to act to get there. It should also improve as you move to increasingly difficult levels.

Humans already scored 100% on ARC-AGI-3. As of June 2026, the top AI systems on the leaderboard scored below 1.5% [28]. This gap shows an inherent weakness of current AI in adaptive learning.

Reinforcement Learning is a natural solution to this type of problem. In RL, an agent learns by interacting with an environment and getting feedback. This is similar to ARC-AGI-3, where the agent has to learn from its own experience, though the existing RL agents are performing poorly on this benchmark. StochasticGoose, the best RL-based solution yet, managed to achieve about 12% on the preview competition [26]. Whereas the main competition's top score is only 1.21%.

The RL agent in this paper was used on the ARC-AGI-3 games following the game's rules, where it has to pass a game level without many tries, and its performance is almost similar to a random agent. Only when the agent is forced to play a single level multiple times does the agent's performance tend to increase. However, this enhancement in execution is not a sign of quick adaptation; rather, it shows that the agent is memorizing the level. This behavior depicts that the agent does not have the ability to quickly adapt in an unseen environment, which perfectly demonstrates the primary challenge of ARC-AGI-3. Making a significant contribution to this benchmark is challenging for even RL agents due to three significant reasons. Primarily, there is no dense per-step reward signal. The agent receives no intermediate feedback on whether it is progressing. It only observes a terminal outcome after a sequence of actions. In addition, the settings are entirely new. There is almost no certain way for the agent to anticipate what might occur next so that it can apply knowledge from what it has learned in the past. Lastly, learning must be done from scratch in every episode. For standard RL methods, this is very challenging.

The aim of this thesis is to gain insight and to find solutions to the problem.

To solve these limitations on selected aspects, this research presents ILP as an alternative approach to RL. Unlike RL, which requires multiple repetitions to solve a level with a good score, ILP can generalize an induction purely from a few examples. This quality of ILP makes it more compatible with ARC-AGI-3.

1.3 Problem Statement

The three main challenges that exist for any agent are outlined in ARC-AGI-3.

1.3.1 Goal Discovery

The reward signal is not defined explicitly for the agent. The reward function tells the agent if it is on the right path in the standard RL. The goal must be determined by the agent in ARC-AGI-3. Current RL methods find it very hard to implement this type of unsupervised goal discovery.

1.3.2 World Model Induction

The agent needs to create a model of the environment within itself. It must learn that certain behaviors result in certain consequences. This model should be developed based on the agent's personal experiences only. The environments are developed in a way that each one is fundamentally different from another. As a result, all the previously used pattern-matching techniques do not work effectively. To succeed in each level, the agent has to learn every level by experiencing that level.

1.3.3 Efficiency

Completing a level is not enough. The agent has to find a solution to it using the minimum number of actions. There is a scoring system that awards efficiency. Humans deal with these environments very effectively and rapidly with very few actions. This is more efficient than what current AI systems are capable of.

Table 1.1: Problem decomposition: what each challenge requires and what this paper tests.

Problem	**What the agent must infer**	**What this paper tests**
Goal discovery	Which states count as success	Hidden-goal games with symbolic states
World-model induction	How actions change states	Maze transitions with Popper
Efficiency	How many actions are needed	Maze path length and ARC-AGI-3 context

These three challenges are not mutually independent. The agent needs a goal to make an efficient plan. It is very difficult to find the goal without a world model. Given that the solution to the three challenges is arduous, the principal technical objective of this thesis is to decouple the perception from world model induction and goal discovery by solving each of the challenges independently. This establishes the fact that which part of ARC-AGI-3 is solvable using ILP and which part requires further experimentation.

1.4 Objective

The objective of this research is to methodically analyze the effectiveness of the RL agent on the ARC-AGI-3 benchmark to precisely show the limitations of the

agent. Based on that, propose a solution using ILP which can demonstrate better performance in those sections where the RL agent fails.

To achieve this, the RL agent is placed in the ARC-AGI-3 environment and executes it to find the score it gets on the first try. Then, to test the proficiency of ILP, the perception layer of ARC-AGI-3 is temporarily removed to perform two controlled experiments. Firstly, the Symbolic-RL framework [7]'s ILASP induction method is replaced by Popper [12], and legacy experiments are run to compare the updated agent's results with the Q-learning baseline. A second test is performed to show whether the agent can find out the hidden goal in the artificial environment, which directly addresses ARC-AGI-3's lack of an explicit reward signal and its requirement for agents to infer goals through interaction [27]. Lastly, based on the feedback from those controlled experiments, we introduce our main agent called InteractivePopper to complete the first game of ARC-AGI-3.

Thus, the 4 contributions we make in this paper are:

1. We compare an RL and a random agent on ARC-AGI-3 and show that on ARC-AGI-3, an RL agent is equivalent to a random agent.

2. We propose PermanentLearner, a symbolic RL approach built on top of ILP(RL) [7], which, unlike RL, is able to learn the underlying rules of the game permanently.

3. We test our approach on 3 toy games that embody the spirit of ARC-AGI-3 to some extent and show that PermanentLearner learns the underlying rules of the game in symbolic form with extreme sample efficiency.

4. Finally, we introduce InteractivePopper, which is able to beat ls20 level 1.

1.5 Methodology in Brief

The first trains and tests a Deep Q-Network RL agent inside an ARC-AGI-3 game environment, looking separately at how it performs on a first attempt versus how much it improves with repeated practice. The second grew out of a practical constraint: pulling symbolic information directly out of ARC-AGI-3's 64×64 grid is not within the scope of this study. So instead of tackling that directly, two controlled, scenario-independent test setups were built to probe what ILP can actually do. One reproduces a classic maze-based Symbolic-RL setup; the other is a separate goal-discovery test. Both run on the Popper ILP system, version 3.1.0, as the core learning engine.

1.6 Scope and Limitations

1.6.1 Scope

This paper is strictly dedicated to the ARC-AGI-3 benchmark. It doesn't endeavor to make a universal artificial intelligence system. These controlled, scenario-independent experiments are meant to test how much promise Inductive Logic Programming actually holds as an alternative, not just to explain why RL agents fail

on ARC-AGI-3. This research makes no claim to a full ILP-based solution covering the whole ARC-AGI-3 environment. Instead it pulls apart two core sub-problems, world model induction and goal discovery, and tests each on its own, since solving the full scenario problem just wasn't realistic given the timeline and scope of this study.

1.6.2 Limitations

Limitation 1: No Explicit Reward Signal. Standard RL approaches are not so effective in the face of sparse or implicit rewards. The agent needs to have its own notion of progress, which is a great technical hurdle.

Limitation 2: Genuine Novelty. The benchmark environments do not allow for any pre-training to aid. The agent is not allowed to use memorization or pattern matching. It must learn only during the actual episode at the moment. This is extremely challenging for existing deep learning systems.

Limitation 3: Efficiency. The scoring system of ARC-AGI-3 not only accounts for the success of an agent in a specific level but also considers if the agent took minimum actions to get the success or not. Humans are thoroughly effective at efficiently solving this game environment. However, the current AI systems are yet to be capable enough to solve those games efficiently.

1.7 Layout of the Dissertation

The arrangement of the subsequent chapters is detailed below:
Chapter 2: Literature Review Rigorously examines the papers that are pertinent to ARC-AGI-3 and RL.
Chapter 3: Requirements, Impacts and Constraints Presents what is needed for the project, its social and environmental effects, and potential threats.
Chapter 4: Proposed Methodology Introduces the methodology and the implementation plan of the agent.
Chapter 5: Results Analysis Shows the test results of RL, PermanentLearner, and InteractivePopper.
Chapter 6: Conclusion The main findings, contributions, and directions for future work are summarized in this chapter.

Chapter 2

Literature Review

2.1 Preliminaries

2.1.1 ARC-AGI-3

ARC-AGI-3 is the 3rd benchmark in the ARC-AGI series. The ARC Prize Foundation has published it in March 2026 [27]. It is the first in a series of fully interactive reasoning benchmarks.
ARC-AGI-3 is different from ARC-AGI-1 and ARC-AGI-2, which set up static puzzle benchmarks, in that it puts an agent into the 'live' game environment. No instructions, no rules and no given goal. The agent has to learn the environment without any instruction and determine how the environment is controlled, what the objective is, and how to accomplish it effectively.

Structure of the Games

There are several game environments in the benchmark. There are at least 6 levels in each of the games. New mechanics are added to each level, which the agent will need to learn and understand.
Each game environment includes the following:

- **Observations:** A 64×64 grid that has 16 colors available. This is what the agent will see on each turn.
- **Actions:** The agent can have 7 possible actions. They are: `ACTION1` `ACTION2` `ACTION3` `ACTION4` (directional action: up, down, left, right), `ACTION5` (general interaction: select, rotate, execute etc.), `ACTION6` (click action, x, y coordinates: 0–63), and `RESET` (restart current level).

A standard API provides the agent with access to the game. It is given an observation frame, chooses an action and is told if the state changed at each step. Rewards are only given once a level has been successfully completed. This means that the reward signal is very sparse.
There are 135 environments in total for the benchmark. These include 25 that are publicly available for development. Of the rest, 55 are semi-private, used for the leaderboard evaluation. The rest of the 55 are completely private and are dedicated for the final competition.

Scoring: RHAE

The Relative Human Action Efficiency (RHAE) is the scoring method used in ARC-AGI-3 [27]. Not only does it check if the agent reaches a level, but also whether it does it efficiently or not, in relation to a baseline of human performance.
Scoring is done by the following method:

1. The number of actions required for each level completed is noted.
2. This is contrasted with the upper-median best human action count at that level, which is the baseline for the human.
3. $\left(\frac{\text{Human actions}}{\text{Agent actions}}\right)^2$ is the per-level score. If a human took 10 actions to complete a level, and the agent took 100 actions, the agent gets 1% for the level.
4. The maximum per-level score will be capped at 1.15. Even if an agent is able to find a solution faster than the human baseline, they cannot score more than 115% for one level.
5. The per-game score is a weighted average score of all the per-level scores. Additional levels are assigned greater weights as they are more challenging and involve more in-depth knowledge of the game.
6. The final total score is the average of all of the per-game scores.

It is crucial to have the squared formula. It heavily penalizes agents that complete levels but do so very inefficiently. If an agent takes 10 times more actions than humans then it does not get 10%, it gets only 1%. This design should teach agents to act efficiently, rather than eventually.

World Model Induction

World model induction occurs when an agent learns a mental model of the environment based on interactions with and exploration of the environment [30].
This differs from having a pre-trained world model. World model induction takes place in real time, at the time when the agent encounters a new environment for the first time.
A world model is an answer to questions like: what occurs if I do this action, if I am in this state? Which states can lead to a goal? What happens when you change the input for the environment?
Induction of the world model is deemed as a requirement in ARC-AGI-3. The environments are really new and there is no prior knowledge to be transferred. The agent should learn the world model from the experiences it has within itself in each episode. It is impossible for the agent to plan effectively or find the goal without an accurate world model. This is why current AI systems like the most powerful frontier models only achieve scores below 1% on this benchmark.

2.1.2 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning. Here an agent learns by interacting with an environment. At each step, the agent observes the current state, takes an action and receives a reward. The agent's task is to discover a policy. It maximizes total reward over the course of time.
RL is mathematically formulated as Markov Decision Process (MDP) [2]. MDPs have five components:

- S: All possible states.
- A: All possible actions.
- Reward Function $R(s, a)$: This gives the reward when action a is taken by an agent in state s.
- Transition Function $P(s' \mid s, a)$: The probability of being in state s' given that the action taken in state s is a.
- γ: The discount factor, where $\gamma \in [0, 1)$. It entails the value of the agent for the future reward vs the immediately available reward.

The agent learns a policy $\pi : S \to A$ that maximizes the expected return:

$$G_t = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right] \tag{2.1}$$

Here, G_t is the expected return from time step t, R_{t+k+1} is the reward received at a future time step $t + k + 1$, and γ is the discount factor that controls how much the agent values future rewards relative to immediate ones.

2.1.3 Inductive Logic Programming

Inductive Logic Programming (ILP) is a subfield of machine learning. ILP learns hypotheses in the form of logic programs instead of numerical models such as weight matrices or decision boundaries [3], [14]. In the traditional machine learning approach, data is usually represented in tabular format or as a numeric feature vector. Rather, in Inductive Logic Programming (ILP) knowledge is expressed in the form of logical facts and relationships. This enables ILP to be effective at processing structured information and learn useful patterns even with a limited number of examples.
There are three parts to a formal ILP task (B, E^+, E^-), where:

- B is the background knowledge: contains the facts and relationships that are already known.
- E^+ is a set of positive examples: contains the facts that the learned hypothesis should explain.
- E^- is a set of negative examples: contains the facts that the learned hypothesis should not explain.

Given these inputs, the goal of an ILP system is to find a hypothesis H, which is a set of logical rules. The hypothesis should explain as many positive examples in E^+ as possible while avoiding the negative examples in E^-. The rules in H are usually written in the following form:

$$\text{head} \leftarrow \text{body}_1, \text{body}_2, \ldots, \text{body}_n \tag{2.2}$$

This means that the head is true whenever all the atoms in the body are true. Since B, E^+, E^-, and H use the same logical language, the rules learned by an ILP system can also be added to the background knowledge. This allows the system to reuse knowledge learned from one task in other tasks [14].

The set of all possible hypotheses that can be created from the given background knowledge is called the hypothesis space. In ILP, learning is treated as a search through this space. The system looks for a hypothesis that is logically consistent with the given positive and negative examples.

An important extension of the usual concept-learning setting is program synthesis. In this setting, the positive and negative examples are given as input-output pairs, and the learned hypothesis is an executable program. Cropper and Dumančić (2022) demonstrate this by learning a quicksort program from a small number of sorted and unsorted list pairs [14]. The system uses background relations such as `partition` and `append`. The resulting program can correctly sort lists of different lengths, though it was learned from only a few examples.

Several properties of ILP make it relevant. First, ILP can learn from a small amount of data compared to many statistical machine learning methods. This is useful when an agent has only a limited number of interactions with an environment. Second, ILP produces logical rules that are easier to understand and interpret than the parameters of neural networks. Third, ILP naturally works with relational representations, that allow it to describe relationships between different entities. These relationships can be difficult to represent using fixed-size feature vectors.

These properties are closely related to the requirements of world model induction discussed in the previous section. In both cases, the system needs to learn an accurate and generalizable model of an unfamiliar environment from a limited number of interactions.

Popper

Popper is a modern ILP system based on the learning-from-failures approach [12]. Instead of searching through the entire hypothesis space directly, Popper first generates a possible hypothesis and tests it against the examples. When the hypothesis fails, however, fails to cover enough positive cases, or to cover a negative case, or to include unnecessary rules, Popper uses that information to add constraints. These restrictions reduce the search space that is not likely to be a solution.

This process is repeated in a generate-test-constrain loop. Popper uses Answer Set Programming (ASP) to implement this process, that helps it search efficiently through a large number of possible logic programs.

Like other ILP systems, Popper takes background knowledge and positive and negative examples as input. It follows the same (B, E^+, E^-) structure described earlier. In Popper, these are provided in three separate files. The examples file contains the positive and negative examples using `pos/1` and `neg/1` facts. The background

knowledge file contains additional relations that Popper can use when creating a hypothesis. The third file is the bias file, which is specific to Popper. It defines the hypothesis space by specifying which predicates can appear in the head of a rule (`head_pred`) and which predicates can appear in the body (`body_pred`), along with their arities.
Popper is designed to support learning recursive programs. In this, the same predicate can be both in the head and body of a rule. This can be enabled by adding `enable_recursion` to the bias file. Popper also supports predicate invention, that allows it to create new intermediate predicates when they are needed to build a hypothesis. The bias file can also include optional type declarations and direction declarations. Type declarations control which predicates can share variables, and direction declarations specify which arguments should be provided (`in`) and which can remain unbound (`out`). These settings can reduce the search space by removing invalid or unusable rules before they are tested.
Popper can also learn from noisy data. When the `--noisy` option is enabled, the hypothesis does not need to perfectly match all the examples. Instead, Popper looks for a hypothesis with a low MDL (Minimum Description Length) cost. This cost considers both the complexity of the hypothesis and the number of positive examples it fails to explain or negative examples it incorrectly explains. This makes Popper more suitable for real-world data, such as game or sensor data, where some examples may be incorrect or inconsistent [12].

2.2 Review of Existing Research

2.2.1 ARC-AGI-3 Preview Agent Competition

The ARC-AGI-3 Preview Agent Competition was from July 18, 2025 to August 19, 2025 [19]. The competition was run by the ARC Prize Foundation, in collaboration with Hugging Face, and lasted for 30 days. Teams competed to create agents that can solve 3 private game environments from the ARC-AGI-3 preview set. All scores and codes were made public.

1st Place: StochasticGoose

Developer: Dries Smit **Score:** 12.58% **Levels Completed:** 18

Approach. StochasticGoose was a purely RL-based approach [26]. The author selected RL because the reward structure of ARC-AGI-3, which is only one reward signal per level, is well suited to the RL framework. LLM-based approaches were identified and rejected. The image-based observations (64×64 grids) and the very long episode lengths were incompatible with the use of LLMs within the time frame of two weeks.
The most difficult aspect of the technical issue was the size of the action space, particularly `ACTION6`, which involved selecting x,y coordinates from a 64×64 grid. To solve this, the agent was designed to use a CNN which would accept the frames as an observation and then predict what actions were likely to affect the game state. In `ACTION6`, a spatially-aware decoding strategy was used that was more stable than flat prediction strategies.

For the sake of sample efficiency, all level frame transitions were held in a memory buffer and then used for off-policy learning. Duplicate pairs of frame-action were avoided by using hash tables. The model was trained periodically and reset between levels.

Limitations. The strategy used was based on semi-random exploration and some brute-force techniques. The author noted that this method would be less effective in future ARC-AGI-3 games when they become resistant to brute-force solutions. The agent also made some simplifying assumptions such as assuming that the complete game state is observed in the observation frame.

2nd Place: Blind Squirrel

Developer: Will Dick **Score:** 6.71% **Levels Completed:** 13

Approach. Another strategy employed by Blind Squirrel was RL [21]. One important aspect of this solution was frame segmentation, which involved partitioning the visual observation into meaningful segments before making decisions on actions. This made it easier for the agent to explore the space and look at only the parts of the frame that could be acted upon as opposed to treating the entire 64×64 grid as one. This segmentation strategy may have been more beneficial to StochasticGoose's performance if used, the first place author noted.

Limitations. The solution was not the top one, indicating that frame segmentation was not enough to cope with the complexity of the private game environments.

Honorable Mention: Explore It Till You Solve It

Approach. A research paper published on this solution, *Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks*, employed a training-free approach of graph-based exploration [25]. It did not employ RL or LLMs. Instead it used vision-based frame processing, in conjunction with systematic state-space exploration.
The agent first decomposed the visual frames into meaningful components and marked the regions of actionability based on visual salience. Then it maintained a directed graph of all the explored states and transitions. For each step, the agent selected the action that led it to an untested state-action pair that had the shortest path. This enabled it to systematically rather than randomly investigate the state space.
This method cleared a median of 30 levels on the preview challenge across six games and ranked 3rd on the private leaderboard. This significantly beat frontier LLM-based agents.

Limitations. It is training-free, which means that it does not learn from experience. It becomes randomly explorative when the status bar is shown in the target game integrated into the scene instead of at the edge. The method becomes virtually intractable with large state spaces, such as game `ft09` levels 3–4. The method also does not leverage previous experiences across levels, and thus cannot get better with the passage of time.

2.2.2 Symbolic Reinforcement Learning using Inductive Logic Programming

Kunii (2018) proposed ILP(RL), a proof-of-concept that pairs ILP with RL to learn state transitions in grid maze environments [7]. Rather than learning a value function over states and actions, the agent works out a hypothesis describing how the environment moves from one state to the next, and an ASP planner then uses that hypothesis to string together a sequence of actions toward a terminal state.
ILASP2i does the induction work. At each step the agent watches a state transition happen and turns it into a context-dependent positive example, following the CDPI format from Section 2.1.3. If the current hypothesis can't account for a new example, ILASP gets called again to patch it up. Once the agent finds a terminal state and has a hypothesis good enough to work with, Clingo solves an ASP program to produce an action plan.
Testing happened in simple VGDL-based grid mazes across 30 independent runs of 100 episodes each, 250 steps per episode. ILP(RL) hit the optimal policy by around episode 40 on average; Q-learning took until roughly episode 60. Each ILASP call took 5.58 seconds on average in the baseline environment, with about 12.83 calls needed per episode. Adding a teleport link as a new transition concept pushed the search space from 1690 candidate rules up to 32755, and average induction time jumped to 95.47 seconds. In the transfer experiments, a hypothesis learned in one maze carried over cleanly to a structurally similar one, letting the agent plan from episode 0 with no re-induction needed.

Limitations. Everything here was tested in small, deterministic, discrete grid mazes only. Induction was the main bottleneck, and it slowed the whole approach down even on simple problems. The framework also needed a hand-written language bias before it could learn anything, assumed the agent could see adjacent cells, and couldn't run ASP planning until a terminal state had actually been found — which cuts against goal-directed MDPs where rewards are sparse.

2.2.3 The ILASP System for Inductive Learning of Answer Set Programs

Law et al. (2020) walk through the ILASP system in detail, covering its learning framework, the kinds of examples it accepts, and how its algorithms changed across four versions [10]. ILASP learns Answer Set Programs instead of Prolog programs, which opens up a lot more ground to work with. Most ILP systems only learn definite rules, but ILASP can handle normal rules, choice rules, and both hard and weak constraints, so it stretches to problems like preference learning, common-sense reasoning with defaults and exceptions, and theories that aren't strictly deterministic.
An ILASP task has three parts: background knowledge B, a mode bias M that sets the rule space S_M, and a set of examples E. The task is to find a hypothesis $H \subseteq S_M$ where $B \cup H$ covers every example in E, preferring the shortest H that does the job (Occam's Razor, essentially). ILASP works with three kinds of examples. Positive and negative examples describe partial interpretations that either should or shouldn't come out as answer sets of the learned program. Context-dependent

examples go further by giving each example its own local background knowledge, a context C, so the property gets checked against $B \cup H \cup C$ instead of just $B \cup H$. Ordering examples, which come into play for learning weak constraints, express a preference between pairs of answer sets rather than saying whether something exists at all.
The paper traces the system through four versions. ILASP1 and ILASP2 are batch learners, encoding the whole task as a meta-level ASP program that Clingo solves; ILASP2 beats ILASP1 by pulling out violating reasons instead of enumerating every violating hypothesis, which gives it orders-of-magnitude speedups on tasks with negative examples. ILASP2i tackles the problem of large example sets by keeping only a small relevant subset around and interleaving the program search with a search for examples still uncovered. The experiments in the paper put it at over two orders of magnitude faster than ILASP2 on tasks with hundreds of examples. ILASP3 is built for noisy data: it assigns penalties to examples and minimises $|H|$+cost, translating each relevant example into coverage constraints on the solution space so the program search only needs a single-shot ASP call rather than several. None of these versions scale well against large rule spaces, and the authors flag that as the main bottleneck still standing, and where their ongoing work is headed.

Limitations. As documented by Law et al., all four ILASP versions construct the full rule space before learning starts, and that construction cost is the core scalability bottleneck [10]. The paper points to FastLAS [9], a related system built alongside ILASP, which sidesteps this by computing only an OPT-sufficient slice of the rule space. Though for now it only works on non-recursive, observational predicate tasks. The wider gap between what ILASP can express and how well it scales to large, real-world rule spaces is still an open problem.

2.2.4 Duck Harness

Duck Harness is an open-source agent harness for ARC-AGI-3 built by Tufa Labs. It landed among the top entries in the ARC-AGI-3 Kaggle competition [29]. Instead of leaning on reinforcement learning or a symbolic model of the environment, it treats each game as a coding problem and let it lean on whatever problem-solving ability a pre-trained Large Language Model (LLM) already has.
The setup drops the LLM into a Read-Eval-Print Loop (REPL) where every game observation shows up as a Python variable. The model pokes at these variables through tool calls, calls on pre-built helper functions, and issues actions that get executed in the ARC-AGI-3 environment. Each action comes back with an updated frame, which refreshes the relevant variables in the sandbox for the next turn. Frames reach the model as both image and text, and the interaction history stays inside the context window by dropping the oldest messages once things get too long. Kaggle's semi-private and private tracks only allow inference on a single GPU, so the authors were stuck working with small open-source models that don't push much token throughput. What they report is built around Qwen 3.6 27B FP8, tested on the 25 public ARC-AGI-3 games with 20 attempts each, landing a mean score of 1.6002 and a standard deviation of 0.4475 [29]. Swapping in GPT-5.4 and comparing against the Executable World Models agent, the two harnesses ended up solving roughly the same set of games, but Duck Harness got there at about a tenth

of the cost per game. With the harness architecture held fixed, then, which games get solved comes down mostly to the underlying model's capability, while what it costs to solve them comes down to how the harness itself is built [29].
Duck Harness fits a wider pattern showing up across the strongest current ARC-AGI-3 approaches: treat the benchmark as a code-generation and tool-use task for an LLM, rather than as a classical sequential decision-making problem for model-free reinforcement learning to grind through. This framing gets around the reward-sparsity problem that motivates the ILP-based approach in this thesis, but it does so by leaning on the world knowledge and general reasoning already baked into a large pre-trained model, not on any explicit, inspectable model of environment dynamics learned from interaction.

2.2.5 Deep Q-Networks

Deep Q-Networks (DQN) were introduced by Mnih et al. [5]. It is the first deep learning model that learned control policies directly from raw-pixel inputs via RL without any handcrafted features or feature extraction. Previous RL techniques involved manually designed features, and applied them to visual input. However, when using more complex models, Q-learning would be unstable and sometimes may not converge during learning. In contrast, DQN did this using Q-learning and a convolutional neural network, which was trained through stochastic gradient descent.
The network is getting the raw pixel data and it is returning a value function that estimates the future rewards, but it doesn't have any game-specific information, or any look into what's going on in the emulator. Each frame is converted to gray-scale, down sampled and cropped to an area of 84×84 and the final 4 frames are combined to create the input to the network. The architecture is very simple. It has two convolutional layers followed by a fully connected layer. The last layer has one output for each potential action and thus the network is able to compute the value of all actions in a single pass.
Experience replay is used in DQN for stability in training. The agent learns from all of these experiences and will randomly draw from its memory for training the network. This will keep the training stable and not have two experiences be similar to each other. In training, actions are taken according to an ϵ-greedy policy (where ϵ is decreasing linearly over time) to ensure a balance between exploration and exploitation.
The DQN was evaluated in 7 games from the Arcade Learning Environment (ALE) for the Atari 2600. The same architecture and hyperparameters are applied to all games, and no tuning is done per-game. It performed better than the previous RL methods in six of seven games and than an expert human in three of seven. It demonstrates that a single general-purpose agent can directly learn effective policies without any domain specific feature engineering at all [5].
The catch, however, is that DQN relies on a dense, frequent reward signal to inform its value function and the reward signal from each game only provides feedback after a few actions. This is where it gets in the way in reward-less environments such as ARC-AGI-3, where there is no per-step score and feedback may not be received until after a long time or in some instances, not at all.

2.2.6 Summary of Key Findings

Table 2.1: Comparison of ARC-AGI-3 Preview Agent Competition approaches.

Approach	**Observation handling**	**Learning strategy**	**Main limitation**
StochasticGoose Score: 12.58% Levels: 18 [26]	CNN over 64×64 frames	RL with replay and action prediction	Semi-random exploration; assumptions about full state observability
Blind Squirrel Score: 6.71% Levels: 13 [21]	Frame segmentation	RL	Segmentation alone insufficient for private environments
Explore It Till You Solve It Score: — Levels: 30/52 [25]	Visual decomposition	Training-free graph exploration	State-space growth makes it intractable in large environments
PermanentLearner Score: this thesis	Structured symbolic states	Popper + ASP planning	No perception layer; small controlled environments only

Distinct from all the other agents in Table 2.1, PermanentLearner was examined using a controlled environment rather than on ARC-AGI-3. It is added here solely for the purpose of showing the contrasts of learning strategies against the competition entries. None of the top three teams used an LLM-based solution. Two of them relied on RL; the third was a structured search method with no learning involved at all. That's telling: it suggests current LLMs simply can't cope with ARC-AGI-3's image-based, long-horizon demands, while pure RL and brute-force exploration hit their own hard ceilings. The top score in the main competition sits at just 1.21%, nowhere near what teams pulled off in the preview round.

That gap between the preview and the main competition says a lot about how much harder the main conditions actually are. Figuring out exactly where and why these approaches break down is the central question this paper tries to answer.

Earlier work on ILP applied to interactive environments gives useful grounding here. Kunii (2018) showed that an ILP agent built on ILASP2i can learn a correct state transition model from relatively few interactions and reach an optimal policy faster than Q-learning in simple grid mazes [7], with the learned hypothesis carrying over to structurally similar environments without needing to be re-induced. But induction itself was the bottleneck, scaling badly as the hypothesis space grew, and the whole setup depended on hand-written language bias plus a discrete, deterministic environment. Law et al. (2020) trace how ILASP evolved across versions and point to the size of the rule space as the scalability wall that none of them have gotten past [10]. Taken together, these two works make the same point from different angles: ILP can build interpretable, generalizable world models from a handful of examples, but scaling that up to large, unstructured observation spaces is still unsolved — which is precisely the wall ARC-AGI-3's 64×64 grid observations put up.

2.2.7 Failure Analysis of Existing Approaches

Both the preview competition results and our own experiments point to the same conclusion: existing methods struggle on ARC-AGI-3, and for a handful of overlapping reasons.

Rewards are scarce. An agent only gets signal once it clears a level, with nothing in between, so it has no way to tell whether what it just did helped or hurt.

The state space compounds this. Observations come as a 64×64 grid over 16 colours, and `ACTION6` requires picking coordinates out of that entire grid, a lot of ground for any agent to cover.

Solving a level also takes many sequential steps, and standard RL methods have a hard time tracing success or failure back to the specific early actions that caused it.

And generalization barely exists: every game in the benchmark is its own thing, so whatever an agent learns in one doesn't carry over to the next. It's starting from zero each time.

Cracking any one of these bottlenecks would likely move the needle on the benchmark.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

3.1.1 Technical and Methodological Requirements

The system is based on ILP and ARC-AGI-3 benchmark environment. The main engine is based on Learning from Failures (LFF) framework, and is a version of Popper v3.1.0, committed at `227c30f` [12]. Popper takes hypotheses as input in Python 3 and calls SWI-Prolog to test them, and it is the Answer Set Programming (ASP) search in the hypothesis space performed by the solver `clingo` that is the interesting part.
For each ILP task, there are 3 files: bias file (`bias.pl`) to describe the hypothesis space, bk file (`bk.pl`) to contain known relations, and exs file (`exs.pl`) to contain positive and negative examples. The 64×64 grid observations need to be converted to logical atoms that ARC-AGI-3 can use; this is a crucial stage in the pipeline for the remaining components.

3.1.2 Software Requirements

- Python 3
- Popper v3.1.0, pinned at commit `227c30f` [12]
- SWI-Prolog
- Clingo (Answer Set Programming solver)
- OpenAI Gym `0.26.2`, Pygame `2.6.1`, py-vgdl
- ARC-AGI-3 API [19]

3.2 Societal Impact

ARC-AGI-3 tests a kind of general intelligence that current AI systems still struggle with [27], so progress on it feeds directly into research on more general systems. One

main difference between the ILP approach and neural methods is that ILP produces logical rules that people can understand. Instead of giving only model weights, it can show how it reached a result. This is useful when we need to understand why an AI made a certain decision.
ILP can also learn from a small number of examples. The maze experiments in this thesis show this clearly. In the Phase 2 smoke test, Popper learned an eight-rule transition model from 28 recorded transition examples and produced a valid seven-step plan. In the 30-seed Phase 3 baseline, total induction time averaged 6.23 seconds per seed. These are separate measurements from different experimental phases and reflect how little data and compute the symbolic approach requires compared to most modern AI methods.
That inspectability also matters for safety. Because the rules an ILP system learns can be read directly. Someone with domain expertise can check them for sense and fix them if they're wrong. It is useful wherever a system needs scrutiny before deployment.

3.3 Environmental Impact

Training modern AI models, particularly those of large neural networks, is a power guzzler [27]. This thesis needs much less computing power. Popper is a symbolic system which may be executed on a common CPU, without the need of a GPU. Rather than adjusting millions of parameters, it scans through the logical programs that it can think of and tests them in order to discover a successful one.
This is reflected in the experiments. No GPU acceleration anywhere, all this was to be done on standard CPU hardware. An average induction time of 6.23 seconds per run in the 30-seed baseline (Table 5.3) indicates that the learning of symbols is helpful and is not a process that requires huge computing infrastructure. Since an ILP agent can construct a working world model through a small number of interactions, it could require much less computation than reinforcement learning methods that normally require millions of interactions with the environment before they converge.

3.4 Ethical Issues

There are a few ethical issues to consider in this project. First, the ability to work in new and unfamiliar environments is part of the wider goal of general-purpose AI. So, it is important to clearly understand what the system can and cannot do before using it in real-world situations.
Interpretability is another important issue. Popper produces logical rules that people can read and check. For example, the eight-rule transition model from the maze experiments can be viewed directly in Prolog. If one of the rules is wrong, it can be changed more easily than trying to understand or modify the internal weights of a neural network. This can be useful in areas where safety is important.
Third, ILASP2i, the induction engine used in the original baseline paper, could be launched but the real symbolic-rl context-dependent tasks failed with a `Poco::WriteFileException` in this environment. Popper was therefore evaluated as a replacement using an incremental collect/check/re-induce loop; the compiled Popper

task and bias are not byte-for-byte identical to ILASP2i's search space. This thesis reports that limitation openly wherever it's relevant.
Finally, ARC-AGI-3 has limits as a proxy for intelligence [8]. A system can do well on one benchmark without demonstrating anything general, and there's a real risk of a system just learning to game the benchmark itself. What this thesis shows should be read as evidence about performance on ARC-AGI-3 and its maze domain, not as a direct measurement of general intelligence.

3.5 Standards

This project follows standard machine learning research practice: reproducibility, open-source code, and transparent reporting of results. Popper enforces deterministic execution through fixed random seeds (`PYTHONHASHSEED=0`, `clingo --seed=1`), which is what makes the reported results reproducible. Every experiment is pinned to a specific software version: Popper v3.1.0 at commit `227c30f`, Gym `0.26.2`, Pygame `2.6.1`.
Development follows general software engineering practice in line with IEEE 730 (Software Quality Assurance). All code, configuration files, and result artifacts will be released publicly once the thesis is submitted.

3.6 Project Management Plan

3.6.1 Resource Management

Every tool used here is open-source, so there's no software licensing cost. All experiments ran on standard CPU hardware. No GPU or cloud compute was ever procured.

3.7 Risk Management

Table 3.1: Risk Register

Risk	**Likelihood**	**Mitigation**
Some of Popper's scalability limits on large ARC-AGI-3 state spaces	High	Experiments were scaled down to use only the `aaa_small` maze and hidden-goal templates; ARC-AGI-3 grid observations simplified before induction.
No other software runs in the current environment	Confirmed	Used published ILASP report values as reference (used Popper as a functional replacement on a documented bounded protocol).

Continued on next page

Table 3.1: Risk Register (Continued)

Risk	Likelihood	Mitigation
Real ARC-AGI-3 API unavailable for the PermanentLearner experiments	Confirmed scope boundary	Controlled symbolic experiments (Contributions 2–3); InteractivePopper (Contribution 4, Section 5.8) uses the download toolkit to run games offline.
Learned hypothesis fails to cover all cases	Medium	Coverage checked after each new transition; re-induction was triggered when coverage fails under the incremental collect/check/re-induce protocol; failed seeds reported separately.

3.8 Economic Analysis

This is a very inexpensive method. All used, including the Popper, SWI-Prolog, Clingo, the supporting Python libraries, are open source, and none of the experiments required cloud compute or a GPU.

There are other approaches similar, but using RL, which were the ones that won the ARC-AGI-3 preview competition. StochasticGoose [26] required long training time with CNN based action prediction with long episode horizons, which is a workload which tends to scale poorly as the state space increases. In the maze domain, the ILP approach used here required approximately 4 seconds of induction time per seed, and no replay buffer to manage, as well as no gradient updates.

The actual cost of this project is research time: design the system, write it up and build it. There were no external sources of funding or hardware required.

Chapter 4

Proposed Methodology

4.1 Methodology Overview

In this chapter we have described our four experiments. The first one is a standard RL agent that is tested on one ARC-AGI-3 level to see how it performs without repeated practice (Section 4.2). The second one is PermanentLearner. It is an ILP-based method that is tested on a small maze to check if it learns movement rules faster than Q-learning (Section 4.3). Thirdly, PermanentLearner is tested on three toy games where the goal is never given. We wanted to check if it can also find a hidden goal on its own (Section 4.4). Lastly, InteractivePopper combines these ideas into a single agent and plays an actual public ARC-AGI-3 game (Section 4.5). Each step handles one subproblem of the main problem before moving to a harder and more realistic setting. The subproblems are: world-model learning, goal discovery, and real gameplay.

4.2 RL vs. Random Agent Methodology

We test on only the first level of `ls20`, one of the 25 public games of ARC-AGI-3 [27]. We use a simple reward scheme: a +1 reward for when the agent experiences the win-condition, and 0 reward for all actions other than that. We use a DQN [5] agent from Stable-Baselines3 [13] with a custom CNN. See Table 4.1 for hyperparameter values.

Table 4.1: Hyperparameter Specifications and Network Architecture for DQN on ARC-AGI-3

Category	**Hyperparameter**	**Value / Specification**
Environment & Input	Observation Space Dimension	$1 \times 64 \times 64$
	Palette Value Range	$\{0, 1, \ldots, 15\}$ (Categorical)
	Normalization	Disabled (`normalize_images=False`)

Continued on next page

Table 4.1: Hyperparameter Specifications and Network Architecture for DQN on ARC-AGI-3 (Continued)

Category	Hyperparameter	Value / Specification
	One-Hot Indicator Channels	16
Feature Extractor (CNN)	Convolutional Layers	3 layers ($16 \to 32 \to 64 \to 64$)
	Kernel Size, Stride, Padding	3×3, Stride 2, Padding 1
	Spatial Pooling	Adaptive Average Pooling to 4×4
	Flattened Feature Size	1,024 ($64 \times 4 \times 4$)
	Latent Projection (d)	256 units (`Linear` + `ReLU`)
Q-Network Head	Architecture	2-layer MLP
	Hidden Units	$[64, 64]$
	Hidden Activation	ReLU
	Output Layer	Linear ($64 \to \lvert\mathcal{A}\rvert$)
DQN Algorithm & Training	Optimizer	Adam
	Learning Rate (α)	5×10^{-4}
	Discount Factor (γ)	0.99
	Replay Buffer Capacity (N)	30,000
	Batch Size (B)	64
	Warmup / Learning Starts (T_{start})	1,000 steps
	Update Frequency	1 gradient step per 4 env steps
	Target Network Update Interval (C)	250 steps

4.3 PermanentLearner Methodology

The interaction of the environment, first stated in terms of state transitions and then followed by inducing hypotheses based on those transitions and then using the hypotheses in an ASP planner to generate action sequences, and then repeating the cycle of interactions with the environment is the same in PermanentLearner as in ILP(RL) [7]. The name is based on this central concept: In standard RL the "learning" from one level isn't transferred to be stored in generalizable knowledge, but rather requires a lot of repetition each time (as shown in Contribution 1). In contrast, the induced rules in PermanentLearner are explicitly represented in symbolic form, and upon successful learning do not need to be learned again. Its greatest attribute is its persistence.

In the original ILP(RL) system [7], they had used ILASP2i [10] as the induction

server. This system was not able to be run reliably in our experimental environment, so ILASP was replaced by Popper [12] in PermanentLearner, with the rest of the pipeline (ASP planner and evaluation methodology) remaining the same to facilitate a fair comparison with the original design. To make the replacement possible, three important changes were made to the examples of the ILASP: (a) Each example had to be translated into a flat example, stating whether the goal is achieved or not; (b) The action in each example had to be a relation with a direction, e.g. `step_up`, `step_down`, `step_left`, `step_right`, which means that the action and the direction of the movement are not conflicting; and (c) The examples contained an "open" cell, but this one had to be represented as a positive predicate instead of stating "not wall", because Popper's bias is not the same as ILASP's.

PermanentLearner's key learning process is an iterative one: the next experience is captured, and the existing hypothesis is tested to see if it can explain the experience, only invoking Popper if the hypothesis doesn't explain the experience. This same general approach without restricting it to any particular domain, is used for both transition rule induction and goal induction (see Section 4.4). Q-learning is maintained as the baseline from the original model to isolate the effect of changing the induction backend only, and observe it.

4.4 PermanentLearner Experimental Setup on 3 Toy Examples

One of the main defining features of ARC-AGI-3 [27] is the agent receives no explicit goal or reward signal. The agent has to discover the rules and its own objective through interacting with the game environment.

We wanted to test if PermanentLearner's induction method can handle this task. We designed three small symbolic toy games. We deliberately set aside ARC-AGI-3's unsolved perception problems. These games have no explicit goal concept in a controlled setting.

The games gradually grow complex according to their target predicate

Unary game: the goal depends on a single object's property for example: an object of a specific color must be there.

Relational game: the goal depends on a relationship between two objects for example: two similar objects will share the same attributes.

Compositional game: the goal combines multiple conditions together.

The agent is never told about the goal predicate, it only sees whether a state is a success or a failure. The agent uses the coverage check then reinducts the loop from section 4.3. By doing this it builds a hypothesis about the hidden goal predicates from its observations.

We have tested the learned rules later in a held out set of unseen states. This experiment ensures that the agent is not memorizing the rules rather it's actually generalizing them. We built these held out states using the same method but we kept them out of the induction phase.

We gave the learned goal predicate directly to the ASP planner. This step proved that goal discovery is not just a classification exercise; it showed that the agent can actually plan and reach its own goal using what it learned. With this we completed the loop from raw interaction to planned action.

Each game was run on multiple independent random seeds to confirm consistent results. Our main sample-efficiency metric was the number of examples needed for the hypothesis to stabilize.

This metric shows a contrast with contribution 1. We mentioned standard RL needed thousands of repeated steps to make a small progress on a single level. Here our symbolic approach learns the goal with a very few examples.

4.5 InteractivePopper

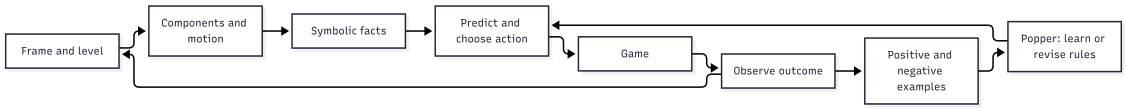


Figure 4.1: InteractivePopper Workflow

Finally, we were able to build an agent that interacts with a real ARC3 game: it played and completed ls20 level 1 within 59 actions. We call it InteractivePopper. InteractivePopper starts with some background knowledge and some predefined assumptions.

4.5.1 Predefined Assumptions and Background Knowledge

Predefined Assumptions

1. It uses connected-component labeling for vision. So, it assumes that adjacent tiles of same color constitute a single object.

2. It can model the 64x64 grid using only 2 square channels at a time.

Background Knowledge and Bias

InteractivePopper uses some pre-given hints as background knowledge. For example, the concept of an opposite point is given to InteractivePopper as a fact. It uses this concept of oppositeness later to induce the rotation rule in ls20.
Moreover, the bias file specifies that rules can have at most 3 conditions and 4 variables, so that the search for Popper is easier.

4.5.2 InteractivePopper Workflow

1. At time step t, it receives only three things from the environment: the 64x64 grid, the current level of the game from the ARC game, and the current actions it has available. It receives no explicit goal.

2. It learns what its actions do by trying them out one by one. For example, it stores its current coordinate, presses ACTION1 (say right), and then receives its next coordinate in $t+1$. Then, it subtracts the initial coordinate from the final to figure out that ACTION1 corresponds to "RIGHT".

3. The agent compares consecutive frames, and detects the region where there are pixel changes. Then a bounding box of size NxN is snapped to that region. It records which pixels were occupied and which were not before the move.

4. Now, if the goal is known, the agent just uses BFS. But if the goal is not known, the agent uses curiosity to select its action. It prioritizes states which are unvisited.

5. The chosen action is sent to the environment, and executed.

6. Observe outcome.

7. InteractivePopper generates positive and negative examples on the fly, where active pixels are labelled positive, and blank pixels negative.

8. When predictions don't match with the real state of the environment, Popper is invoked. Popper induces the rule and then feeds it back to step 4. Go to step 1.

Chapter 5

Result Analysis

5.1 Performance Evaluation

In this section we have evaluated the performance of the approaches based on efficiency, accuracy, reliability, and test results. One of the main purposes of this evaluation is to understand the failure of the original approach of reinforcement learning (RL) with ARC-AGI-3. The analysis also focuses on symbolic learning and whether it can address some of these problems.

5.2 RL vs. Random Agent Results

Our first experiment focused on the behavior of the RL agent on ARC-AGI-3. The per-seed results are shown in Table 5.1, the verified recordings in Table 5.2, and the learning curve in Figure 5.1. The RL agent performed the same as a random agent when it was given to complete a game level within a limited number of steps. Though its performance got better when it was given the chance to repeatedly play the same level. It is notable, as this repeated exposure gives the agent a chance to remember the earlier actions and use them later in the gameplay. So this minimal improvement should not be considered as a quick adaptation to a new game environment. Our original thesis also focuses on this memorization behaviour rather than quick adaptation.

This result is related to one of the main difficulties of ARC-AGI-3, the agent does not receive a clear reward in every step. The agent by itself has to discover the goal and understand the environment through interaction. This benchmark also requires the agent to build an understanding of a new environment by its own experience.

5.3 PermanentLearner Initial Results

Secondly we have used Popper to learn the transition rules of a symbolic maze environment. Our goal for this experiment was to determine if the agent is able to learn a general world model from observed state transitions, and later use that model for planning.

In an initial smoke test, the agent collected 28 transitions from the `aaa_small` maze, and Popper learned an eight-rule transition model that produced 28 true positives,

Table 5.1: Comparative Performance Across 5 Independent Seeds on ARC-AGI `ls20` (Level 1, 60,000 Actions)

	Random Agent Baseline		**DQN Agent (Linear ϵ-Decay)**		
Run / Seed	**1st Win Step**	**Total Wins**	**1st Win Step**	**ϵ at 1st Win**	**Total Wins**
Seed 1	53,596	1	15,798	0.53	479
Seed 2	14,150	1	24,755	0.26	6
Seed 3	55,442	1	18,528	0.44	83
Seed 4	47,254	2	8,772	0.74	311
Seed 5	27,073	1	13,797	0.59	167
Mean	**39,503.0**	**1.20**	**16,330.0**	**0.51**	**209.20**
Std. Dev.	±18,097.0	±0.45	±5,908.4	±0.18	±188.50
Min	14,150	1	8,772	0.26	6
Max	55,442	2	24,755	0.74	479

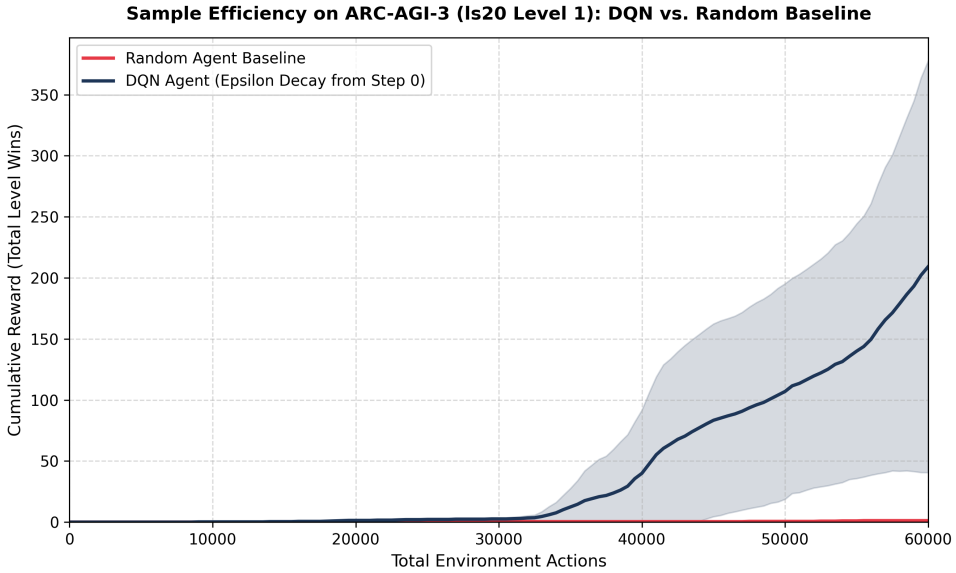


Figure 5.1: Sample efficiency on ARC-AGI-3 (`ls20` Level 1): Cumulative level completions over 60,000 environment interactions comparing the Random Agent Baseline against the DQN Agent across 5 independent random seeds. The solid lines represent empirical means, and the shaded regions indicate ±1 standard deviation.

Table 5.2: Verified ARC-AGI Online Recordings on `ls20` (Human Baseline $H = 22$ Actions)

Recording	Milestones Unlocked	Actions	RHAE	Public ARC Scorecard
1	Milestone 1: First Completion	132	2.78%	`69084f04-34a3...`
2	Milestones 2–4 (25%, 50%, 75%)	23	91.49%	`e3ac1ce9-2736...`
3	Milestone 5: Achieve 100% RHAE	**15**	**115.00**%	`3f9516ff-8d3f...`

112 true negatives, 0 false negatives, and 0 false positives against held-out transitions. This model let the planner reach the goal in 7 steps.

We then evaluated Popper against a Q-learning baseline over 30 independent seeds, using 100 episodes of at most 250 steps each. Table 5.3 reports the results.

Table 5.3: PermanentLearner vs. Q-learning baseline on `aaa_small` (30 seeds)

Evaluation Metric	PermanentLearner	Q-learning
Task Performance (Deterministic Maze)		
Success Rate	1.00 ± 0.00	1.00 ± 0.00
Mean Episode Reward	4.00 ± 0.00	4.00 ± 0.00
Mean Solution Steps	7.00 ± 0.00	7.00 ± 0.00
Learning Dynamics & Computational Cost		
Convergence (Episodes)*	$\mathbf{1.27 \pm 0.51}$	6.93 ± 1.12
Induction Calls	4.60 ± 0.71	—
Induction Time (s)	6.23 ± 1.32	—

* Convergence is defined per backend: for PermanentLearner, episodes until hypothesis convergence; for Q-learning, episodes until the evaluation policy matches the optimal 7-step path.

The convergence metrics are not identical. PermanentLearner's value records the last episode that triggered re-induction, whereas Q-learning's value records the first evaluation matching its final policy. The per-episode traces are therefore the more direct sample-efficiency evidence.

The current results also tell us the Popper system used an average of 4.60 induction calls per run. The total average induction time was 6.23 seconds with a spread of 1.32 seconds. It reached the convergence point after an average of 1.27 episodes and has a spread of 0.51 episodes. This indicates that the symbolic transition model was formed very early during the learning phase, though the induction process itself required computation.

The Q-learning control was also tested with 30 independent seeds using the same number of episodes and steps. Q-learning also achieved a success rate of 1.00 and mean reward of 4.00 and a mean episode length of 7.00. So we can conclude that

both the models produced the same task performance on this simple deterministic maze.

An earlier two-template pilot reported convergence after an average of 5.5 examples (unary) and 3.4 examples (relational), with all runs reaching 100% agreement on held-out states and no false convergence. The final three-game benchmark below supersedes this pilot.

The final hidden goal experiment combined goal learning with planning across three symbolic games — unary, relational, and compositional — each evaluated over 10 seeds. Table 5.4 reports the results.

Table 5.4: Hidden-goal induction and planning results, 10 seeds per game

Game	**Ex. to conv.**	**Held-out**	**Precision**	**Recall**	**Plan succ.**	**Plan len.**
Unary	2.500±0.500	1.000	1.000	1.000	1.000	3
Relational	5.600±4.964	1.000	1.000	1.000	1.000	6
Compositional	5.600±4.176	1.000	1.000	1.000	1.000	4

Because the generators are small and structured, these perfect scores demonstrate pipeline feasibility rather than performance on the full ARC-AGI-3 problem.

This result shows that the symbolic system was able to learn both the environment dynamics and the hidden objectives in the tested symbolic environments. These results are especially important for the hidden goal experiment, as the agent was not informed about its actual goal. It had to learn the rules from the interaction data and then used the learned rule inside the planner.

5.4 Analysis of Design Solutions

Our experiment shows three main stages of the developed solution.

The first stage was the direct RL approach. This approach was useful for identifying the main limitations of the problem. The agent's behavior and performance were similar to that of a random agent when it was deployed in a new level with a limited number of attempts. The agent performed better when the same level was repeated many times. This shows that repeated experience helped the agent to reuse its previously gathered information from that same level. But it did not show any sign of quick adaptation that is required by ARC-AGI-3.

The second stage was symbolic transition learning. The agent learned rules to describe how an action changes the current state instead of trying to learn which action produces high value. The learned transition model has eight rules that cover the movement in all four directions and the behavior when the agent is blocked by a wall.

This design had a very high accuracy in the tested maze. In part two of the experiment, all 28 transitions were explained by the learned model, and it also rejected all 112 negative cases. No false positives or negatives were recorded. This learned model could be converted to planner representation and used to produce a valid seven-step plan.

The strength of this design is that the learnt knowledge is explicit; the result is a set of logical rules rather than a massive set of numerical values. This human-readable format makes the model easier to inspect and understand. Use of symbolic rules allows the same type of knowledge to be reused later.

The main weakness of the model is the cost of induction. Popper has to search through possible logical rules, which takes additional computation. In the 30 seed baseline experiment, the average induction time was 6.23 seconds. Symbolic learning reduces the need for repeated trial and error methods, but it does not make the learning process computationally efficient.

The third stage was the hidden goal learning; the agent was not given any kind of direct goal in this design. It had to observe the symbolic states and the outcomes of its actions to learn a logical rule to describe the goal.

The result shows that this design was highly accurate for the tested goal types. All three hidden goals of the games achieved 100% agreement, 100% precision, and 100% recall. All the runs across the three types of games also achieved successful planning. This implies that the learned goal rule was able to classify the states correctly and was usable by the planner to complete the task.

This hidden goal experiment also tells us that the agent did not require a large number of examples in these small environments. The average point of convergence was between 2.5 and 5.6 examples, depending on the types of the goals. This is consistent with the motivation for using inductive logic programming in these small, structured environments; it does not establish general sample-efficiency or scalability. A hypothesis can be learned from a relatively small number of structure examples. Our thesis also describes ILP as a method that can learn logical rules from limited data.

This symbolic approach was practical for the small symbolic environments in terms of feasibility and cost. Our experiments were run on normal CPU-based resources and did not require any neural network training or GPU acceleration. But our current result does not prove that this computational cost is applicable for all the games of ARC-AGI-3.

Overall the design experiments show that symbolic learning addressed two important parts of the problem separately. The transition learning provides a world model, and hidden goal learning provides a description of the objective. By separating the problems it is easier to inspect and give the planner explicit knowledge to work with.

5.5 Final Design Adjustments

The final design was adjusted based on the results obtained from the experiments.

The first adjustment was to replace ILASP2i with Popper. The original symbolic RL framework was based on ILASP but the ILASP2i system couldn't be run in our current environments due to compatibility issues and its problems in execution. So we introduced Popper as the symbolic induction backend.

The second adjustment that we made was on the representation of the transition rules. Original ILASP formulation allowed the direction of an action and the direction of an adjacency relation to be mismatched. This could result in a logically

valid rule but it's not a valid movement rule. To solve this, we updated the system to use direction-specific background relations like `step_up`, `step_down`, `step_left`, and `step_right`. This forces the action and movement direction to align.

Our third adjustment was the explicit representation of open cells. The original transition rules used the absence of a wall as a part of the rule. But the Popper implementation represents the local components of the wall explicitly using "open" which allows Popper to use the same logical idea without depending on unsupported negation behavior.

Fourth adjustment was to separate transition learning from goal learning. The transition learning experiments first focused on understanding how actions change the environment. The hidden goal experiment then focused on learning what state should be considered successful. This separation made it possible to study world model induction and discover goals independently.

For our fifth adjustment we used held out examples for goal learning experiments. The agent learned the goal rule from one set of interaction examples and later was tested on new examples that were not used during the induction process. With this we were able to provide the agent with direct tests to check the learned rules and generalise beyond training examples.

Our last adjustment was to use the learned symbolic rule directly inside the existing ASP planner. The learned rule is not only used for classification but it becomes a part of the planning process and allows the agent to generate an action sequence toward the learned goal.

By introducing the aforementioned changes we managed to produce a more controlled symbolic RL framework. The final design focuses on learning symbolic transition and goal rules from experience and using those rules for later planning.

5.6 Statistical Analysis

The analysis here is descriptive.
For the baseline comparison between Popper and Q-learning, each method ran on 30 independent seeds. Results are reported as mean and population standard deviation across success rate, mean reward, mean number of steps, induction time, number of induction calls, and convergence episode.
The independent goal-learning experiments used 10 seeds per goal template, tracking the number of examples needed to converge and agreement on held-out states. Experiments were also run across three game types using 10 random seeds each, measuring learning speed, accuracy, and planning success.
Formal statistical tests such as t-tests or p-values were not used. This was intentional. The goal was not to prove a statistical trend but to verify whether the system could correctly learn the rules and successfully solve problems in small, controlled environments. Running multiple seeds was meant to test stability across different random starting conditions. The zero spread on success rate, reward, and step count across all 30 baseline runs shows that the final evaluation result was completely stable in this particular maze.
The goal-learning experiments produced the same pattern, with zero spread on held-out agreement, precision, recall, and planning success. Every seed led to successful

classification and planning in these symbolic games. That stability is real, but it should not be read as proof the same will hold in larger or noisier environments. These were small, controlled setups, and stronger general conclusions would require formal statistical testing on a larger and more varied benchmark.

5.7 Comparisons and Relationships

The first comparison sets the original RL behavior against the symbolic approach. In the RL experiment, performance on a new level was close to random when the agent had only a few attempts, and it improved mainly through repeated exposure to the same level. This points to the agent relying on repetition to reuse information from one environment rather than adapting quickly on its own.

The symbolic approach builds explicit rules from observations instead of learning by trial and error. Rather than memorising one exact sequence of actions, it learns general rules that can be applied to solve new instances of the same problem.

When tested on a simple, deterministic maze across 30 seeds, both Popper and Q-learning produced identical outcomes: a 100% success rate, a mean reward of 4.00, and a mean path length of 7 steps. Since both performed perfectly, neither method outperformed the other on this metric alone.

Where they differ is in how they got there. Popper's hypothesis converged after a mean of 1.27 episodes; Q-learning needed a mean of 6.93 episodes to reach its optimal evaluation policy. These figures are not directly comparable, since they measure different things: one is symbolic hypothesis convergence, the other is when the learned policy reached its final evaluation behaviour.

The third comparison looks back at the earlier ILP-based RL work reviewed in Section 2.2. That work, using ILASP2i [7], had the agent reach optimal policy around episode 40 against Q-learning's episode 60 in a similarly simple maze, and showed that symbolic transition rules could transfer to a structurally similar environment, though induction itself was identified as a major bottleneck. The current Popper experiments follow the same core idea of learning an explicit symbolic transition model and planning from it. Given the different backend, protocol, data collection, and evaluation conditions, however, these results should not be read as a numerical replication of that earlier work.

The fourth comparison sits between transition learning and hidden-goal learning. Transition learning shows the system can learn how the environment changes after an action; hidden-goal learning shows it can also determine which state counts as a goal. Together they cover two complementary halves of the overall problem.

The hidden-goal results go a step further by showing that the learned goal rule connects directly to planning: all three tested game types achieved 100% held-out agreement and 100% planning success. The goal induction step held up in actual decision-making, not just as a classification exercise.

The transfer experiment adds one more perspective on symbolic knowledge reuse. A transition model learned on `aaa_small` was carried over to the `experiment3_after` maze, and across five seeds the agent achieved 100% success with a 21-step plan every time. This shows the learned transition representation could support planning in a different registered maze environment. The original target, `aaa_medium`, was unavailable due to a missing level file, so `experiment3_after` served as the closest available alternative. This result is therefore better read as a limited transfer

experiment than a full evaluation of general transfer ability.
Both methods reached the same score on this simple maze. The real difference lies in what they learned: the symbolic system did not just solve the puzzle; it identified and expressed the underlying transition rules in an inspectable form along the way.

5.8 InteractivePopper Results

For our fourth experiment we tested InteractivePopper, a combination of pixel/movement rules generated by Popper, a hand-crafted empirical table of action displacements, and breadth first search on the learned model. This agent does not employ an ASP planner, and the system as a whole should not be considered as being entirely ILP based. Tested on real public ARC-AGI-3 games (`ls20`, `ft09`, `cn04`) and the game files were downloaded using the official ARC Prize download toolkit, but the games were not played as a live scorecard session (Table 5.5).

On `ls20`, the 1st level is finished on both runs (59 actions and 60 actions, respectively). If we base it on the human level (Table 5.2) it comes to an RHAE of about 13.9% and 13.4% on the two seeds, respectively, which is a genuine, if not substantial, achievement. These two runs then went on to Level 2 and applied the rules they had just learnt in this level without re-induction, but the agent failed to select an action in this level: in the controlled synthetic corridor experiment (Section 4.4), the same planning branch had succeeded. The agent could not finish any level of the other games (here `ft09` and `cn04`) within the budget of the action (Table 5.5).

Table 5.5: InteractivePopper performance in public ARC-AGI-3 games

Game	**Action of 1st Level Clear**	**Total Actions**	**Levels Completed**	**Outcome**
`ls20` (seed 0)	59	126	1	Game over, Level 2
`ls20` (seed 1)	60	127	1	Game over, Level 2
`ft09`	—	80	0	Budget exhausted
`cn04`	—	60	0	Budget exhausted

The percentage of regions that change (Table 5.6) is also significantly lower than the overall percentage, because many of the registered regions didn't change from step to step, which introduces a bias into the overall statistics. These results are not an evaluation performed blindly, without trying to take into account what the developers were looking for, but the results of a system which was being iteratively modified as it progressed, in response to observing `ls20` and the other games that were public. No ablation or RL comparison was made in order to isolate the specific contribution of Popper from that of the hand built displacement table.

Table 5.6: InteractivePopper prediction accuracy

Game	Position Correct	Bitmap Correct	Correct on Changing Regions
`ls20` (seed 0)	113/119	166/168	12/14
`ls20` (seed 1)	115/120	162/164	6/8
`ft09`	—	110/113	1/2
`cn04`	—	0/1	0/1

5.9 Discussions

The results provide us with evidence that the main difficulty in the tested environments is not just choosing actions; the agent also needs to understand how the environment works and identify what counts as success. The original RL experiment addressed this issue as it behaved the same as a random agent when it was given to solve a new level with limited attempts. It became better after repeated exposure to the same level.

The symbolic experiment provides a different approach to solve this problem, i.e., instead of expecting a single learning mechanism to discover everything at once, the problem was divided into smaller segments. The transition learner was used to build a model environment, and the hidden goal learner was used to discover the objective. The learned rules were then given to an ASP planner that could use them to generate action sequences.

In a small deterministic environment, the correct symbolic world model can be produced based on the transitions. The eight learned transition rules correctly described the observed movements and blocking behavior in the tested maze. So the planner could reach the goal in only seven actions.

The hidden-goal agent achieved perfect held-out agreement, precision, recall, and planning success across the three tested game types. These goal learning results are also important, as they show that goal discovery can be expressed as a symbolic induction problem when the state is already presented in a structured format.

The learned output can be easily read and inspected by humans, which are sets of logical rules. This interpretability is different from learning only numerical value functions as symbolic rules describe the relationship that the agent has learned. Our thesis also identifies this interpretability and the ability to learn from limited examples as an important property of ILP.

This result also shows us the cost of the symbolic approach; Popper requires time to search for a consistent hypothesis and the transition experiment requires several induction calls in the larger evaluation. The agent needs fewer trial-and-error interactions to understand its environment compared to standard RL. The tradeoff is computational speed, as the environment adds more objects and actions, and the number of possible rules expands rapidly. Searching through these rules requires heavy computation making it the slowest part of ILP.

InteractivePopper's result (Section 5.8) has connected to a real ARC-AGI-3 game. It completed `ls20` Level 1 using the same kind of learned movement rules. It shows

us the symbolic approach is not limited to toy settings. Though it could not clear Level 2 after reaching there. The result portrays a development-informed run rather than a blind test. So we should see it as an early, modest result but not as a benchmark-level one.

Several limitations persist in the current results. Firstly the transition and the hidden goal experiments use small and structured environments. The symbolic states are already in logical form so the experiments do not solve the complete perception problems of ARC-AGI-3.

Secondly, the Popper implementation follows the original collect/check/re-induce pattern used by ILASP2i. But its context-reified task and compiled bias are not a byte-for-byte reproduction of ILASP2i's search space. The results show that Popper works end-to-end as a replacement induction backend, though not as an exact one-to-one substitution.

For our third limitation, the hidden goal experiments were performed on synthetic symbolic environments. The 100% results show successful learning for the tested generators but it is not guaranteed that the same level of accuracy can be achieved on much larger and more complex ARC-AGI-3 game levels.

Fourth, the transfer result is limited because the intended `aaa_medium` registration could not be used, so we chose a different maze, `experiment3_after`. As there were no cross-maze Q-learning transfer comparisons, there are no conclusions on comparative transfer performance from the experiments.

Fifth, the current Popper backend is focused on the ordinary four-action transition setting. So the updated transition does not claim support for more complex transition types like links or teleportation. These cases require additional rules and further testing.

Lastly, the statistical analysis is descriptive. The experiments show very stable results in the tested settings, but we need to do larger experiments on more complex environments to pass a claim about efficiency, scalability, and generalization.

The results support the main idea of the new approach, i.e., symbolic learning can be used to separate world-model induction from goal discovery and then connect the learned knowledge to planning. Our current experiment proves that our idea is effective in a controlled environment, and the rules that are learned can be very accurate, interpretable, and usable for action planning. But these results should be treated as a proof of concept rather than taking it as evidence that the complete ARC-AGI-3 problem has been solved.

Chapter 6

Conclusion

6.1 Summary of Findings

This study has clearly demonstrated the difference between the two approaches. In the first experiment where both agents play each level for the first time, we observe that both the DQN agent and the random agent have near zero win rates until up to 60 k steps have been trained. The randomly selected agent won only an average of about 1-2 times over the entire run. It is not until thousands of repetitions of the same level that the DQN agent began to differ from the random agent. Regardless, those figures were quite different from seed to seed, from 6 to 479 wins. This is not indicative of rapid learning. Rather, it proposes that the DQN agent was predominantly memorizing the same level after trying several times.

The same was not true for the Popper-based symbolic approach. Both in the task of world model induction and in hidden goal discovery, it could learn correct and general rules from just a few examples. The transition learning experiment taught Popper a model of eight rules from 28 observed transitions. No false positive or false negative; the model correctly explained 112 negative and 28 positive cases. Obviously, it also enabled the planner to achieve the objective with just seven steps. In 30 different seeds, Popper was able to discover the right rules in an average of 1.27 episodes, while Q-learning required on average 6.93 episodes. At the end of the day, both methods were equally successful, rewarding, and step-by-step.

The extended hidden goal experiments consisted of three kinds of games: (1) unary, (2) relational, and (3) compositional. In each case, the system obtained 100% held-out agreement, precision, recall, and planning success. It required just 2.5–5.6 examples and 3–6 planning steps. In the limited transfer experiment, the learned transition model was also successfully applied in the alternative maze `experiment3_after`. This highlights the potential for transfer of the learned model outside of the environment in which it was learned.

In our fourth test we checked if InteractivePopper can use these symbolic ideas on a public ARC-AGI-3 game instead of a controlled toy setting. It completed Level 1 of `ls20` in both test runs (59 and 60 actions, about 13–14% RHAE) and reached the next level. But it couldn't clear Level 2. From this result we understand that this approach can connect to real gameplay on the actual benchmark, not only on the controlled experiments.

Overall, the results demonstrate that the ILP-based method can learn world models

and find goals with fewer examples than regular reinforcement learning in controlled and scenario-independent environments. It's also easier to learn since the acquired rules can easily be scrutinized. However, these results are to be considered proof of concept, rather than the full answer. This approach has not yet been used to solve the entire set of problems in ARC-AGI-3, which are presented in the form of scenarios.

6.2 Contributions to the Field

Table 6.1: Summary of Contributions

Sl No.	Contribution	Key Result
1	RL vs. random-agent baseline on ARC-AGI-3 (`ls20`)	A standard DQN agent performed the same as a random agent without repeated practice (Table 5.1).
2	Replaced ILASP2i with Popper in an existing symbolic-RL framework	Verified working substitution, keeping the same ASP planner and Q-learning baseline for fair comparison. Popper reaches a correct hypothesis in 1.27 episodes vs. Q-learning's 6.93 (Table 5.3).
3	Hidden-goal discovery using ILP with no reward signal given	100% held-out accuracy across three game types by using only 2.5–5.6 examples per goal (Table 5.4).
4	InteractivePopper: symbolic agent tested on real, public ARC-AGI-3 games	Completed `ls20` Level 1 in 59 to 60 actions with 13–14% RHAE. Reached but did not clear Level 2. `ft09` and `cn04` not solved within the action budget (Table 5.5).

6.3 Recommendations for Future Work

The main work for the future will be to resolve the perception problem. The current method of this paper cannot be executed on ARC-AGI-3's [27] raw 64x64 grid due to its limitations.

Secondly, the current Popper method's bounded induction loop should carefully follow the ILASP-based online iterative loop; that's why it should extend to be a true online version. Thirdly, both world model induction and the goal discovery method should be evaluated in more complex, diverse, and large environments to verify the scalability, particularly including complex or coordinate-based action and teleportation or link mechanics to extend the transition backend. Fourthly, to enlarge the domain of this empirical investigation, depending on the descriptive statistics is not enough to get reliable data; rather, formal statistical tests should be done.

Therefore, if the perception problem becomes solvable, goal discovery and transition learning can be incorporated into a unified agentic loop; this approach can create

a path to solve the combination of all three challenges, i.e., goal discovery, world model induction, and efficiency together.

Finally, InteractivePopper was not tuned for all the games, so future work needs to test it on those unseen environments. Then Popper needs to be separated from the hand-built displacement table to isolate its contribution, and an investigation should be done to find out why it worked on the controlled test (Section 4.4) but could not take any action at the game's Level 2.

Bibliography

[1] J. McCarthy, M. L. Minsky, N. Rochester, and C. E. Shannon, "A proposal for the Dartmouth summer research project on artificial intelligence," Dartmouth College, Hanover, NH, Tech. Rep., Aug. 1955, Original proposal submitted to the Rockefeller Foundation. [Online]. Available: https://raysolomonoff.com/dartmouth/boxa/dart564props.pdf

[2] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, no. 1, pp. 9–44, 1988. DOI: 10.1007/BF00115009

[3] S. Muggleton, "Inductive logic programming," *New Generation Computing*, vol. 8, no. 4, pp. 295–318, 1991. DOI: 10.1007/BF03037089

[4] S. Pinker, *The Language Instinct, How the Mind Creates Language*. New York: HarperPerennial, 1995, ISBN: 0-06-097651-9.

[5] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. DOI: 10.1038/nature14236

[6] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961

[7] K. Kunii, "Symbolic reinforcement learning using inductive logic programming," M.S. thesis, Imperial College London, Sep. 2018. [Online]. Available: https://github.com/921kiyo/symbolic-rl/blob/master/report.pdf

[8] F. Chollet, *On the measure of intelligence*, 2019. DOI: 10.48550/arXiv.1911.01547 arXiv: 1911.01547 [cs.AI].

[9] M. Law, A. Russo, E. Bertino, K. Broda, and J. Lobo, "Fastlas: Scalable inductive logic programming incorporating domain-specific optimisation criteria," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 2877–2885. DOI: 10.1609/aaai.v34i03.5678

[10] M. Law, A. Russo, and K. Broda, "The ILASP system for inductive learning of answer set programs," in *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, 2020. DOI: 10.48550/arXiv.2005.00904

[11] J. Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, vol. 588, no. 7839, pp. 604–609, 2020. DOI: 10.1038/s41586-020-03051-4

[12] A. Cropper and R. Morel, "Learning programs by learning from failures," *Machine Learning*, vol. 110, no. 4, pp. 801–856, 2021. DOI: 10.1007/s10994-020-05934-z

[13] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

[14] A. Cropper and S. Dumančić, "Inductive logic programming at 30: A new introduction," *Journal of Artificial Intelligence Research*, vol. 74, pp. 765–850, 2022. DOI: 10.1613/jair.1.13507

[15] J. Abramson et al., "Accurate structure prediction of biomolecular interactions with AlphaFold 3," *Nature*, vol. 630, no. 8016, pp. 493–500, 2024. DOI: 10.1038/s41586-024-07487-w

[16] *ARC-AGI-1*, 2024. DOI: 10.48550/arXiv.1911.01547 arXiv: 2412.04604 [cs.AI].

[17] A. Karpathy. "Jagged intelligence: The word i came up with to describe the (strange, unintuitive) fact that state of the art LLMs can both perform extremely impressive tasks...," X (Twitter), Accessed: Feb. 6, 2026. [Online]. Available: https://x.com/karpathy/status/1816531576228053133

[18] Royal Swedish Academy of Sciences, *Press release: The Nobel Prize in Chemistry 2024*, NobelPrize.org, Accessed: 2026-01-27, Oct. 2024. [Online]. Available: https://www.nobelprize.org/prizes/chemistry/2024/press-release/

[19] ARC Prize Foundation, *ARC-AGI-3 preview agent competition*, ARC Prize Foundation, Accessed: 2026-06-08, 2025. [Online]. Available: https://arcprize.org/competitions/arc-agi-3-preview-agents

[20] F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard, "Arc-agi-2: A new challenge for frontier ai reasoning systems," *arXiv preprint arXiv:2505.11831*, 2025. DOI: 10.48550/arXiv.2505.11831

[21] W. Dick, *ARC-AGI-3 agents: Blind squirrel*, GitHub, Accessed: 2026-06-08, 2025. [Online]. Available: https://github.com/wd13ca/ARC-AGI-3-Agents

[22] Y. Huang and L. F. Yang, *Winning gold at imo 2025 with a model-agnostic verification-and-refinement pipeline*, 2025. DOI: 10.48550/arXiv.2507.15855 arXiv: 2507.15855 [cs.AI].

[23] T. Luong and E. Lockhart, *Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad*, Google DeepMind Blog, Accessed: 2026-01-27, Jul. 2025. [Online]. Available: https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/

[24] M. R. Morris et al., *Levels of agi for operationalizing progress on the path to agi*, 2025. DOI: 10.48550/arXiv.2311.02462 arXiv: 2311.02462 [cs.AI].

[25] E. Rudakov, J. Shock, and B. U. Cowley, *Graph-based exploration for ARC-AGI-3 interactive reasoning tasks*, 2025. DOI: 10.48550/arXiv.2512.24156 arXiv: 2512.24156 [cs.AI].

[26] D. Smit, *1st place in the ARC-AGI-3 agent preview competition*, Medium, Accessed: 2026-06-08, 2025. [Online]. Available: https://medium.com/@dries.epos/1st-place-in-the-arc-agi-3-agent-preview-competition-49263f6287db

[27] ARC Prize Foundation. "ARC-AGI-3: A new challenge for frontier agentic intelligence," ARC Prize Foundation, Accessed: Sep. 25, 2026. [Online]. Available: https://arcprize.org/blog/arc-agi-3-launch

[28] *ARC prize leaderboard*, https://arcprize.org/leaderboard, 2026.

[29] H. Bessis, J. Cottaar, I. Pressman, A. Smit, M. Tešnar, and S. Viel, *Duck harness: Winning solution for arc-agi-3 milestone 1*, Tufa Labs Research Blog, Jul. 2026. [Online]. Available: https://tufalabs.ai/research/duck-harness/

[30] F. Chollet et al., *Assessing adaptive world models in machines with novel games*, 2026. DOI: 10.48550/arXiv.2507.12821 arXiv: 2507.12821 [cs.AI].